

Warsaw University of Technology

FACULTY OF
ELECTRONICS AND INFORMATION TECHNOLOGY



Institute of Control and Computation Engineering

Bachelor's diploma thesis

in the field of study Computer Science
and specialisation Software Engineering

Web/desktop application (TypeScript) for visualizing and composing
dependency graph between reusable software components

Binh Vuong Le Duc

student record book number 330097

thesis supervisor

mgr inż. Sławomir Niespodziany

WARSAW 2026

Wersja w języku polsko-angielskim

Politechnika Warszawska
Warsaw University of Technology

.....
miejscowość i data
place and date

.....
imię i nazwisko studenta
name and surname of the student

.....
numer albumu
student record book number

.....
wydział i kierunek studiów
faculty and field of study

OŚWIADCZENIE **DECLARATION**

Świadomy/-a odpowiedzialności karnej za składanie fałszywych zeznań oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie, pod opieką kierującego pracą dyplomową.

Under the penalty of perjury, I hereby certify that I wrote my diploma thesis on my own, under the guidance of the thesis supervisor.

Jednocześnie oświadczam, że:

I also declare that:

- niniejsza praca dyplomowa nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
this diploma thesis does not constitute infringement of copyright following the act of 4 February 1994 on copyright and related rights (Journal of Acts of 2021, item 1062) or personal rights protected under the civil law
- niniejsza praca dyplomowa nie zawiera danych i informacji, które uzyskałem/-am w sposób niedozwolony,
the diploma thesis does not contain data or information acquired in an illegal way,
- niniejsza praca dyplomowa nie była wcześniej podstawą żadnej innej urzędowej procedury związanej z nadawaniem dyplomów lub tytułów zawodowych,
the diploma thesis has never been the basis of any other official proceedings leading to the award of diplomas or professional degrees,
- wszystkie informacje umieszczone w niniejszej pracy, uzyskane ze źródeł pisanych i elektronicznych, zostały udokumentowane w wykazie literatury odpowiednimi odnośnikami,
all information included in the diploma thesis, derived from printed and electronic sources, has been documented with relevant references in the literature section,
- znam regulacje prawne Politechniki Warszawskiej w sprawie zarządzania prawami autorskimi i prawami pokrewnymi, prawami własności przemysłowej oraz zasadami komercjalizacji.
I am aware of the regulations at Warsaw University of Technology on management of copyright and related rights, industrial property rights and commercialisation.

.....
(podpis studenta)
(signature of the student)

Web/desktop application (TypeScript) for visualizing and composing dependency graph between reusable software components

Abstract. [TBD.]

Keywords: [TBD.]

Aplikacja web/desktop (TypeScript) do wizualizacji i edycji grafu zależności pomiędzy reużywalnymi modułami oprogramowania

Streszczenie. [TBD.]

Słowa kluczowe: [TBD.]

Contents

1. Introduction 11

 1.1. Motivation 11

 1.2. Problem Statement 11

 1.3. Project Goals and Scope 12

 1.4. Thesis Structure 12

References 13

List of Symbols and Abbreviations 14

List of Figures 14

List of Tables 14

List of Appendices 14

1. Introduction

1.1. Motivation

In the modern embedded systems development landscape, the cost of software exceeds the cost of designing the hardware it runs on [1]. As semiconductor technology advances and customer demand for broader, more sophisticated functionality grows, an embedded application is assembled by dozens of engineers across several teams, each one producing independent modules that recombine into many product variants across the device's lifetime.

The industry's response to this assembly problem is to formalize it: separate the modules from the application that consumes them, and describe their composition in a structure independent of any single module's source code. The *diff* framework, presented by Niespodziany [1], embodies that response for C++ embedded software through data-driven dependency injection: the application's composition is described in a JSON topology file consumed at startup.

While *diff* improves reusability and flexibility, composing a product variant means editing the topology file manually. The developer must ensure its structural correctness, with limited tool support. This flow does not meet the demands of a reliable, repeatable engineering process [2], and these demands grow with the size of the system.

The industry's assembly problem and the framework's own proposal [1] together motivate *diff-forge*: a visual tool that supports managing the dependency graph, and producing JSON file consumed by the existing C++ without modification. Next section sets out the specific gaps that *diff-forge* aims to resolve.

1.2. Problem Statement

Manual maintenance of a *diff* topology file imposes three major gaps along the developer's workflow. Each grows with the size of the system and the number of developers who work with the file.

Lack of module discovery. Modules are distributed across Artifactory repositories, with many modules per repository and many versions per module. To compose a new system, the developer must learn which modules exist, what interfaces they expose, and what parameters they accept. This gap scales with the size of the catalog and the number of new systems assembled.

Lack of graph readability. The dependency graph is encoded as text. To trace which components depend on a given module, or whether one component depends on another directly or transitively, the developer reads the JSON or runs a custom query against it. Every architectural question, from the impact of a change to the presence of a cycle, requires this reconstruction.

Lack of real-time validation feedback. The JSON parser only checks syntax and is not capable of detecting lexical errors within each entry's configuration fields (such as identifiers, dependency references, parameters, ...), or structural errors across the JSON file (cycles, unmet dependencies, interface mismatches). These errors are caught only when the binary starts, in CI, or during code review. A broken build blocks other developers, and responsibility for fixing it often falls to someone other than the original author.

1.3. Project Goals and Scope

The primary goal of this project is to develop `diff-forge`, a cross-platform desktop application that enables developers to visually compose, verify, and export software topologies. The application utilizes modern web technologies including TypeScript, React [3], and Electron [4]. The specific goals of the application include:

- Providing an intuitive drag-and-drop interface for instantiating components from a managed catalog.
- Implementing a real-time validation engine to prevent structural errors before they reach the production environment.
- Ensuring seamless integration with existing C++ development workflows by exporting framework-compliant configuration files.

1.4. Thesis Structure

The following chapters comprise this thesis: Chapter 2 reviews the state of the art in dependency injection and visual programming. Chapter 3 provides an overview of the `diff` framework and its target use cases. Chapter 4 describes the architectural design of `diff-forge`, while Chapter 5 details its implementation. Lastly, Chapter 6 evaluates the tool's effectiveness, followed by concluding remarks in Chapter 7.

References

- [1] S. Niespodziany, “Data-driven dependency injection for embedded software, enhancing reusability with c++”, *International Journal of Electronics and Telecommunications*, vol. 71, no. 4, pp. 1–7, 2025. DOI: 10.24425/ijet.2025.155469.
- [2] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.
- [3] I. Meta Platforms. “React documentation”. (2024), [Online]. Available: <https://react.dev/>.
- [4] O. Foundation. “Electron documentation”. (2024), [Online]. Available: <https://www.electronjs.org/docs/latest>.

List of Symbols and Abbreviations

EiTI – Faculty of Electronics and Information Technology

WUT – Warsaw University of Technology

DI – Dependency Injection

IAiIS – Institute of Control and Computation Engineering

CBSE – Component-Based Software Engineering

GUI – Graphical User Interface

List of Figures

List of Tables

List of Appendices

1. User Manual	15
2. Configuration Schema	16

Appendix 1. User Manual

Instructions for installing and running `diff-forge`.

Appendix 2. Configuration Schema

The JSON schema used for component metadata.